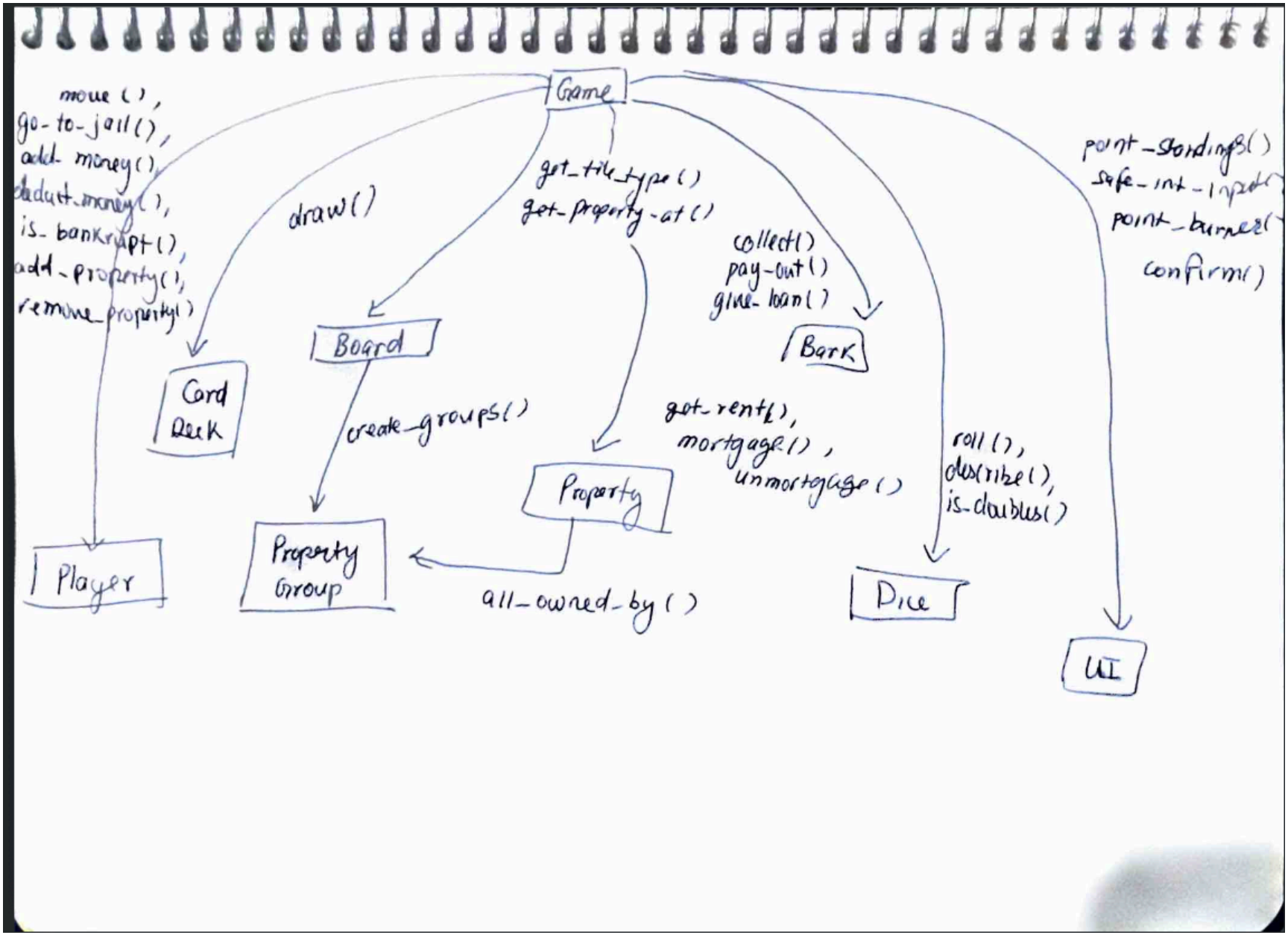
Task 1 Report: White-Box Testing and Code Quality Analysis

1.1 Control Flow Graph

A Control Flow Graph (CFG) was constructed for the MoneyPoly system to represent the internal execution flow of the program. Each node in the graph corresponds to a statement or a block of logically related statements, while directed edges represent the transfer of control between them.

All major components of the system were considered, including player actions, property handling, banking operations, and game flow logic. Special attention was given to decision points such as conditional statements, loops, and function calls, ensuring that all possible execution paths were captured.

The CFG was drawn manually, as required, and attached separately as part of the submission.


Game
move(),
go_to_jail(),
add_money(),
deduct_money(),
is_bankrupt(),
add_property(),
remove_property()
draw()
get_tile_type()
get_property_at()
collect()
pay_out()
give_loan()
print_standings()
safe_int_input()
print_banner()
confirm()
Board
Bank
Card Deck
create_groups()
get_rent(),
mortgage(),
unmortgage()
roll(),
describe(),
is_doubles()
Property
Player
Property Group
all_owned_by()
Dice
UI

1.2 Code Quality Analysis (Pylint)

The codebase was analyzed and improved iteratively using **pylint**. The objective was to enhance readability, maintainability, and adherence to standard Python coding practices.

Initial Analysis

The initial pylint run produced a score of approximately **9.1 / 10**. The following issues were identified:

- Presence of unused variables and imports
- Redundant conditions and unnecessary control flow constructs
- Missing or incomplete docstrings
- Minor formatting inconsistencies

Iterative Improvements

Iteration 1–3: Basic Cleanup and Structural Fixes

- Resolved import path issues to ensure correct module resolution
- Removed unnecessary `else` blocks following `continue` statements
- Eliminated unused variables and redundant conditions
- Standardized formatting and indentation

Iteration 4–6: Refactoring and Structural Enhancements

- Refactored `bank.py` and `property.py` to improve clarity and modularity
- Standardized naming conventions across all modules
- Consolidated jail-related attributes into structured representations
- Improved separation of concerns between modules

Iteration 7–8: Readability and Simplification

- Added missing docstrings to improve code documentation
- Simplified conditional expressions for better readability

- Removed unnecessary f-strings and redundant checks

Iteration 9–10: Final Optimization and Edge Handling

- Fixed edge-case warnings related to exception handling
- Improved initialization logic across multiple modules
- Ensured consistency in function definitions and usage
- Resolved all remaining pylint warnings

Final Result

- Final pylint score: **10.00 / 10**
- The codebase is clean, consistent, and maintainable

1.3 White-Box Testing

A comprehensive white-box testing approach was implemented using **pytest**. The test suite was designed to validate internal logic, cover all execution paths, and identify hidden defects.

Test Coverage

- Total test cases: **90+**

Coverage includes:

- All decision branches
- Key state transitions
- Edge cases and boundary conditions

Testing Strategy

The test cases were designed across multiple dimensions:

1. Functional Testing

- Player movement and position updates

- Property purchase and rent payment
- Trade and auction mechanisms

---

2. State-Based Testing

- Bankruptcy handling and player elimination
- Jail entry, exit, and turn handling
- Mortgage and unmortgage transitions

---

3. Edge Case Testing

- Zero and negative balances
- Empty card decks
- Insufficient bank funds
- Invalid user inputs

1.4 Bug Discovery and Fixes

White-box testing revealed **21 logical and runtime errors** in the system. Each bug was identified through failing test cases, analyzed, and fixed through targeted modifications. Every fix was committed separately to maintain a clear development history.

Detailed Bug List

Error 1: Collection logic skips players with insufficient funds
Players with low balance were ignored instead of being forced to pay.
Fix: Removed conditional check and enforced payment, allowing debt or bankruptcy.

Error 2: Mortgage value printed as method instead of value
A method reference was printed instead of its return value.
Fix: Added parentheses to correctly call the method.

Error 3: Bank crashes when funds are insufficient
The bank raised an exception when funds were low.
Fix: Modified logic to allow negative balance (simulating IOUs).

Error 4: Turn skipping when player is eliminated
Removing a player disrupted turn order.
Fix: Adjusted current index after removal.

Error 5: Dice roll range incorrect (1–5 instead of 1–6)
Dice did not generate valid values.
Fix: Updated random range to include 6.

Error 6: Recursive turn handling caused multiple moves
Recursive calls led to unintended repeated turns.
Fix: Replaced recursion with an iterative loop.

Error 7: Missing pre-turn bankruptcy check
Bankrupt players were still allowed to play.
Fix: Added validation at the start of each turn.

Error 8: GO salary awarded only on landing
Players did not receive salary when passing GO.
Fix: Updated position logic to detect wrap-around.

Error 9: Group ownership logic used `any()` instead of `all()`
Monopoly condition was incorrectly evaluated.
Fix: Replaced `any()` with `all()`.

Error 10: Rent doubled even when property was mortgaged
Invalid rent calculation.
Fix: Added mortgage status validation before doubling rent.

Error 11: Property purchase failed with exact balance
Edge case prevented valid purchase.
Fix: Corrected comparison operator.

Error 12: Rent not credited to owner
Payment was deducted but not transferred.
Fix: Added credit to owner's balance.

Error 13: Bank.collect accepted negative values
Allowed invalid transactions.
Fix: Ignored negative inputs.

Error 14: Loan did not reduce bank balance
Bank funds remained unchanged.
Fix: Deducted amount during loan disbursement.

Error 15: Net worth calculation ignored property values
Incorrect player ranking.
Fix: Included property values in calculation.

Error 16: Unmortgage allowed without sufficient funds
State inconsistency occurred.
Fix: Added affordability check and reverted state if necessary.

Error 17: Jail escape via doubles not implemented
Missing rule from Monopoly mechanics.
Fix: Added dice-based escape logic.

Error 18: Auction implemented as single round
Players could not outbid others dynamically.
Fix: Implemented continuous bidding loop.

Error 19: Trade did not credit seller
Seller did not receive money.
Fix: Added balance update for seller.

Error 20: Player removal broke turn index
Index shift caused incorrect turn sequence.
Fix: Adjusted index after removal.

Error 21: CardDeck crash on empty deck
Division by zero occurred in representation.
Fix: Added explicit handling for empty deck.

1.5 Final Outcome

- Total test cases: **90+ (all passing)**
- Total bugs fixed: **21**
- Final pylint score: **10.00 / 10**
- Complete branch and edge-case coverage achieved

Conclusion

This assignment demonstrates the importance of white-box testing in identifying deep logical flaws that are not visible through surface-level validation. By systematically analyzing the internal structure of the code and designing targeted test cases, critical issues in game logic, state handling, and edge conditions were identified and resolved.

The iterative improvement process, combined with structured testing and clear commit history, resulted in a robust, maintainable, and logically consistent implementation of the MoneyPoly system.